

断点续传 (Resumable Upload) 设计架构

✗ XMLHttpRequest / fetch 本身不能自动续传

✓ 必须“前端 + 后端配合 + 分片上传”才能实现

一、为什么现在做不到自动续传？

现在方式是：整个文件 → 一次 POST → multipart/form-data

问题是无法恢复：

- 网络断了 = HTTP 请求直接失败
 - 已上传的字节 = 后端不会保存“进度状态”
 - 浏览器也不会记住上传到哪了
-

二、标准解决方案：分片上传 (Chunk Upload)

业界标准方案 (百度网盘 / Google Drive 都是这个思路)

✓ 核心思想

把一个文件拆成很多小块：

File (100MB)

↓

Chunk1 (0-5MB)

Chunk2 (5-10MB)

Chunk3 (10-15MB)

...

每一块单独上传

✓ 即使断网：

Chunk1 ✓

Chunk2 ✓

Chunk3 ✗ (断网)

恢复后：继续上传 Chunk3

三、前端实现（核心代码）

1 分片函数

```
function createChunks(file, chunkSize = 5 * 1024 * 1024) {  
  const chunks = [];  
  let start = 0;  
  while (start < file.size) {  
    const end = Math.min(start + chunkSize, file.size);  
    chunks.push(file.slice(start, end));  
    start = end;  
  }  
  return chunks;  
}
```

2 上传分片 + 记录进度

```
async function uploadWithResume(file) {  
  const chunks = createChunks(file);  
  const fileId = file.name + "-" + file.size; // 唯一 ID（实际应后端生成）  
  let uploadedIndex = localStorage.getItem(fileId) || 0;  
  
  for (let i = uploadedIndex; i < chunks.length; i++) {  
    const formData = new FormData();  
    formData.append("file", chunks[i]);  
    formData.append("index", i);  
    formData.append("total", chunks.length);  
    formData.append("fileId", fileId);  
    try {  
      await uploadChunk(formData); // 调用单个 chunk 文件的上传  
  
      localStorage.setItem(fileId, i + 1);  
      const percent = Math.round(((i + 1) / chunks.length) * 100);  
      uploadProgressText.innerText = percent + "%";  
    } catch (err) {  
      console.log("upload paused, network error");  
      // ! 停止上传，等待恢复  
      return;  
    }  
  }  
}
```

```
// 上传完成清除记录
localStorage.removeItem(fileId);

showModal("Success", "Upload completed");
}
```

3 单个 chunk 上传

```
function uploadChunk(formData) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();

    xhr.open("POST", "http://localhost:8080/v1/file/upload/chunk");
    xhr.onload = function () {
      if (xhr.status === 200) {
        resolve();
      } else {
        reject();
      }
    };
    xhr.onerror = function () {
      reject();
    };
    xhr.send(formData);
  });
}
```

四、后端分片接收

1. 接收 chunk

```
@PostMapping("/upload/chunk")
public void uploadChunk(
    @RequestParam("file") MultipartFile file,
    @RequestParam("index") int index,
    @RequestParam("total") int total,
    @RequestParam("fileId") String fileId) {
  // 保存分片
}
```

2. 保存到临时目录

```
temp/  
  fileId/  
    0.part  
    1.part  
    2.part
```

3. 合并文件

当全部上传完成：**mergeChunks**(fileId); 将同一个文件整合成完整文件(Java Random File ?)

五、断网恢复机制

监听网络恢复的情况，设置 **Listener**，页面恢复自动继续上传

```
window.addEventListener("online", () => {  
  console.log("network restored, resume upload");  
  resumeUpload();  
});
```

判断是否有未完成上传

```
function resumeUpload() {  
  const fileId = localStorage.getItem("currentFileId");  
  if (fileId) {  
    uploadWithResume(pendingFile);  
  }  
}
```

六、现在 vs 升级后对比

能力	现在	分片上传
上传进度	✓	✓
网络断点恢复	✗	✓
大文件支持	✗（容易失败）	✓